

Questions

1. (10 points) Identify and justify with appropriate reasoning on the system architecture design from the application component and service delivery perspectives. (hint: refer to the communication and layout of the components)

Each service (Catalogue, Cart, Account, and Order) functions separately and links to a separate database according to the system architecture's modular, service-oriented design. This structure enhances fault isolation, scalability, and maintainability while encouraging the division of responsibilities. via API based interactions, the centralized front end facilitates effective service delivery and streamlines communication by managing client engagements (via online and mobile apps). Clear component duties are guaranteed by this architecture, which also facilitates adaptable updates and service integration.

2. (10 points) With respect to each of the components in the above system architecture, identify and justify with appropriate reasoning the possible security trust boundaries. (hint: refer to your answer in Q1) With their corresponding databases, the Catalogue, Cart, Account, and Order service components in the architecture each create a separate security trust boundary. The purpose of these boundaries is to separate less sensitive processes like product browsing from more critical ones like user accounts and orders. In order to maintain input validation and access control, the front end serves as an extra barrier between internal services and external clients. This method of layered trust boundaries limits unwanted access between components and helps contain possible breaches.

3. (10 points) With respect to the identified security trust boundaries, identify and justify with appropriate reasoning the possible attack surfaces for the given application. (hint: refer to your answer in Q2 and the possible entry/exit points)

Front end open APIs, unprotected routes of communication between services, and possible database vulnerabilities are the primary attack surfaces for the trust boundaries that have been found. For instance, internal services may be at danger from privilege escalation or data leaks due to improperly configured APIs, while the front end may be vulnerable to injection attacks or compromised authentication. To stop attackers from taking advantage of these surfaces and lateralizing across components, entry points such as user inputs and service requests need to be properly secured.

4. (20 points) Perform a vulnerability assessment on the identified attack surfaces and the recommended mitigation techniques for the mapped vulnerabilities. (hint: refer to the widely used open-source resources for vulnerability mapping taxonomies)

I used popular frameworks such as OWASP, CWE, and CVE to map potential risks in order to fix system vulnerabilities. SQL injections, compromised front end authentication, exposed APIs, and improperly configured services are among the main concerns. Input validation, secure authentication, frequent software upgrades, vulnerability scanning, and the least privilege principle are all part of mitigation. By fixing identified flaws and enhancing the overall security posture, these actions aid in system security.